

Cursor使用技巧分享

示例项目仓库地址：

<https://hn.devcloud.huaweicloud.com/codehub/project/759278abbfb14b098eeddc548741f38b/codehub/8146544/home?ref=master>



如遇CodeArts权限问题，找 🐼 宋蒙蒙

如：



引言

本次分享的目标：

- AI Coding 深度用户，人码 AI 三合一，无需了解实践经验 → 了解底层原理；
- 手搓仙人，日码1000+行，不用 AI 依然很舒适 → 进一步提升编码效率，以及提升编码外的效率；
- 团队新人，对项目代码的具体实现不太熟悉，需要边做边学 → 如何快速上手新项目，不出锅；

大家在AI Coding上的常见问题：

- 一口气生成整个需求代码，然后打补丁快？还是边写代码边提问快？
- AI 的能力边界在哪？什么场景下适合使用？什么场景下适合手搓？
- AI 只能帮我写代码吗？能不能探索下新场景？

Cursor 核心特性

四种Agent模式

模式	适用场景	能力	工具
<u>Agent</u>	复杂功能、重构	自主探索、多文件编辑	启用全部工具
<u>Ask</u>	学习、规划、提问	只读探索，无自动修改	仅启用搜索工具
<u>Plan</u>	需要规划的复杂功能	在执行前创建详细计划，并提出澄清性问题	启用全部工具
<u>Debug</u>	棘手 Bug、回归问题	生成假设、日志埋点、运行时分析	全部工具 + 调试服务器

Tab自动补全

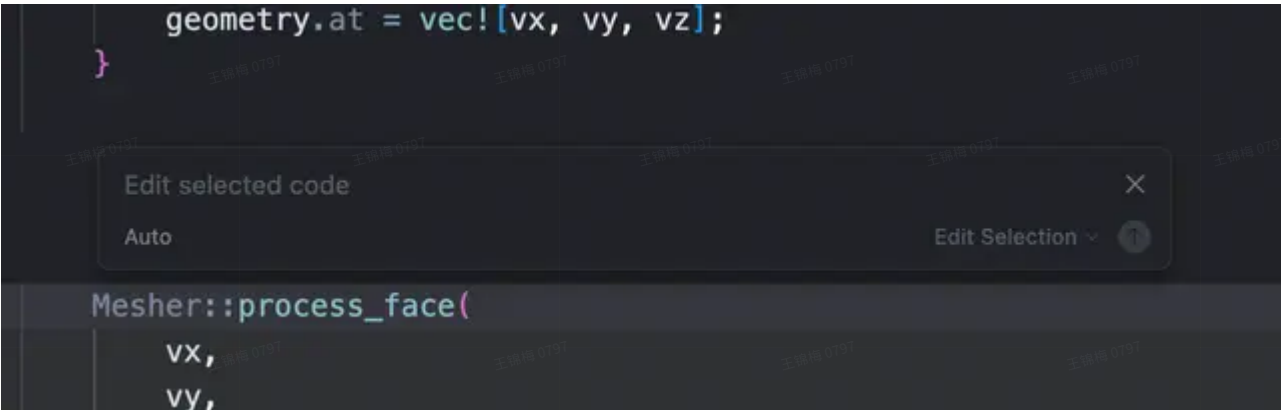
使用Tab进行重复模式的编码是非常高效的

- 一次修改多行
- 自动补全缺失的 import 语句
- 在文件内及跨文件跳转以进行协同编辑
- 基于最近的更改、linter 错误和已接受的编辑获得建议

内联编辑

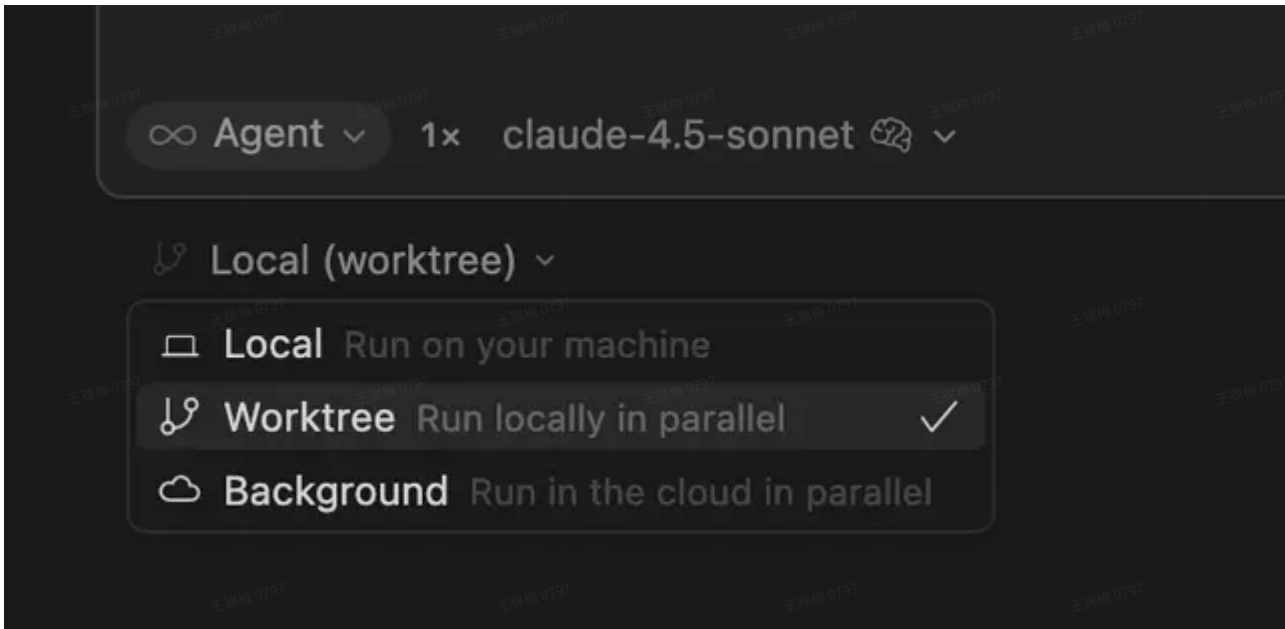
内联编辑功能允许你在编辑器中直接编辑代码或提问。按下 Cmd+K 会打开一个输入框，你选中的代码和指令会一起组成请求。若要对整个文件进行更改，请使用 Cmd+Shift+Return。该模式支持在可控范围内进行全局修改。

在实现原子性功能的时候使用内联编辑很方便，速度快的同时不会消耗额度



并行Agent

并行代理在各自的 worktree 中运行，使它们可以进行编辑、构建并测试代码，且互不干扰。



worktrees 的另外一项强大功能是可以将同一条提示同时运行在多个模型上。在进行复杂问题解决的情况下可以使用这种方法进行多模型效果比对。



模型

默认是Auto模式，会自动根据负载情况选择最可靠最好高级的模型，如果200k的上文还不够用可以切换至Max模式，但不推荐，因为过长的上下文会带来注意力丢失

Cursor 支持各大模型提供商的所有顶尖代码模型。

名称 ^	默认上下文	Max 模式	功能特性
 Claude 4.5 Opus	200k	200k	∞  
 Claude 4.5 Sonnet	200k	1M	∞  
 Composer 1	200k	-	∞ 
 Gemini 3 Flash	200k	1M	∞  
 Gemini 3 Pro	200k	1M	∞  
 GPT-5.2	272k	-	∞  
 GPT-5.2 Codex	272k	-	∞  
 Grok Code	256k	-	∞ 

Cursor上下文

Token的计算机制

LLM推理的效果取决于输入的所有Token

初始 Token 组成： 初始输入 = SystemPrompt + 用户问题 + Rules + 对话历史

用户问题： 我们输入的文字 + 主动添加的上下文（图片、项目目录、文件）

Rules： project rule + user rule

工具调用后的 Token 累积： cursor 接收用户信息后开始调用 tools 获取更为详细的信息，并为问题回答做准备：总 Token = 初始输入 + 所有工具调用结果

实际的上下文示例

代码块

- 1
- [SystemPrompt] 你是一个专业的代码审查助手...
- 2
- [用户问题] 这个函数有什么问题？
- 3
- [图片] [代码截图显示一个Java方法]
- 4
- [项目目录] /Users/KuBo/IdeaProjects/someproject/
- 5
- [当前文件] somemodule/src/main/java/com/xxx/xxx/service/ExampleService.java
- 6
- [Rules]
- 7
- 多模块Maven项目，使用Java 8

```
8 - 始终使用中文回复
9 - 单行不超过120字符
10 ...
11 [对话历史]
12 用户：你好，我想了解一下这个项目的结构
13 AI：这是一个多模块的Maven项目...
14 [工具调用结果1] 文件内容：
15 public class ExampleService {
16     public String processData(String input) {
17         if (input == null) {
18             return null; // 问题：应该抛出异常而不是返回null
19         }
20         // ... 更多代码
21     }
22 }
23 [工具调用结果2] 搜索结果：
24 异常处理最佳实践建议...
25 [工具调用结果3] 语法检查：
26 Line 5: 建议使用Objects.requireNonNull()进行空值检查
```

工具调用

Cursor内置了许多工具，通过工具搭建大模型与外部系统交互的桥梁

 语义搜索

 搜索文件和文件夹

 Web

 获取规则

 读取文件

 编辑文件

 运行 shell 命令

 browser 浏览器

代码库检索

作为一个Coding工具，如何理解代码仓库非常重要，了解这一部分将有助于：

- 提示词的编写：如何提问，编写什么样的 prompt，才能让模型准确地找到它需要的代码？
- 代码规范：在编码时，什么样的代码风格是“模型友好”的？

在用户导入项目时，Cursor 会启动一个 Codebase Indexing 流程，它的进度可以在 Preference - Cursor Settings - Indexing & Docs 中看到。据官方文档描述，这一步主要有 7 个步骤：

1. 你的工作区文件会与 Cursor 的服务器安全同步，确保索引始终最新。
2. 文件被拆分为有意义的片段，**聚焦函数、类和逻辑代码块**，而非任意文本段。
3. 每个片段使用 AI 模型转为**向量表示**，生成能捕捉语义的数学“指纹”。
4. 这些向量嵌入存储在专用的向量数据库中，支持在数百万代码片段中进行高速相似度搜索。
5. 当你搜索时，查询会用**与处理代码相同的 AI 模型转为向量**。
6. 系统将你的查询向量与已存储的嵌入向量进行比对，找到最相似的代码片段。
7. 你会获得包含文件位置和上下文的相关代码片段，并按与查询的**语义相似度排序**。

Cursor的系统提示词

了解Cursor的系统提示词有助于理解其Agent的具体工作原理，并能够识别出其关注的重点是什么，具体的提示词可参考<https://github.com/dontriskit/awesome-ai-system-prompts>，其中也包含了其他著名AI项目的系统提示词

代码块

- 1 Only call tools when they are necessary.
- 2 Before calling each tool, first explain to the USER why you are calling it.

- 必要性原则 + 透明解释：避免滥用工具，同时让用户知道操作原因

代码块

- 1 DO NOT loop more than 3 times on fixing linter errors on the same file.
- 2 On the third time, you should stop and ask the user what to do next.

- 防死循环机制：设置重试上限，避免无限循环修复错误

代码块

- 1 heavily prefer the semantic search tool to grep search, file search, and list dir tools

- 语义搜索优先：利用语义理解提高搜索准确性

代码块

```
1 For ANY commands that would use a pager or require user interaction, append |
  cat
2 The user will have to approve the command before it is executed.
```

- 用户审批机制 + 交互命令处理：确保安全和避免阻塞

Cursor Rules

Project Rule

跟项目绑定的Rule，它的本质是在.git的同级目录下维护一个.cursor的目录，在这里面存放自定义的规则文本，然后在每次会话时根据你的设置，决定要不要把这些内容贴到上下文中。

规则类型	描述
Always Apply	应用于每个聊天会话
Apply Intelligently	当 Agent 根据描述判断其相关时应用
Apply to Specific Files	当文件匹配指定模式时应用
Apply Manually	在对话中被 @ 提及时应用（例如：@my-rule）

使用合适的项目规则可以：

- 沉淀与你代码库相关的领域知识
- 自动化项目特定的工作流或模板
- 统一风格或架构决策

User Rule

跟用户绑定的全局设置，它与项目维度的规则类似，只不过生效范围是全局，它的对应规则文件在用户的根目录下面。需要注意的是，这个规则的更新不是实时生效的，可能要等10分钟左右。

Rules的建议规范

- 将规则控制在 500 行以内
- 将较大的规则拆分为多个可组合的规则
- 提供具体示例或参考文件

- 避免模糊的指导，像写清晰的内部文档那样写规则
- 在聊天中重复使用提示时，复用已有规则
- 通过引用文件而不是复制其内容来使用文件——这样既能让规则保持简短，又能在代码变更时避免规则失效

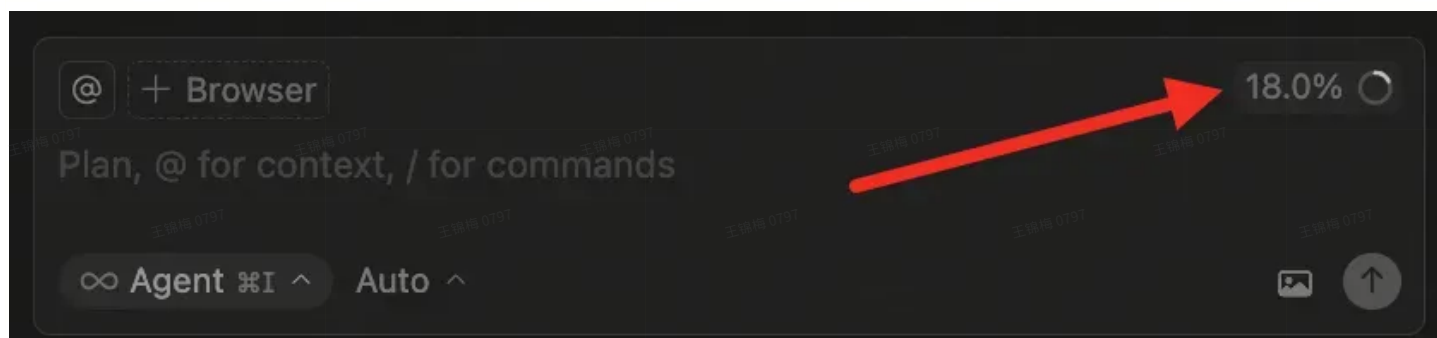
上下文最佳实践

进行清晰的问题描述

在上面的原理部分，我们介绍了模型是如何进行代码库理解的（向量匹配、意图拆解后进行模糊搜索、调用链溯源等）。因此，在描述问题时，我们最好能给出具体的功能、文件名、方法名、代码块，让模型能够通过语义检索等方式，用较短的路径找到代码，避免在检索这部分混杂太多弱相关内容，干扰上下文。

- bad case：请求超时但无报错信息
- good case：我在 `src/components/UploadFile.tsx` 里写了一个文件上传的 `uploadFileToOSS` 异步方法，现在调用后一直报「请求超时但无报错信息」，帮我排查问题并修复，原方法代码如下

把控上下文长度

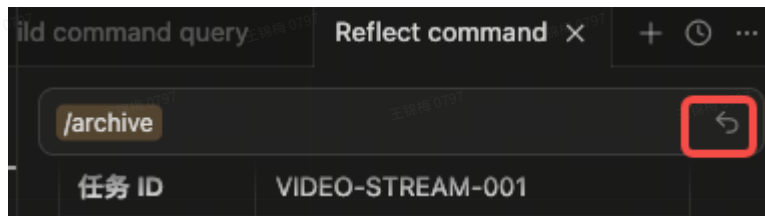


Cursor中能显示当前占用的上下文窗口比例。超出这个比例之后，工具会对历史内容进行压缩，保证对话的正常进行。但被压缩后的信息会缺失细节，所以建议大家在处理复杂问题时，采用上下文窗口大的模型或拆解成小问题，尽量避免压缩。从使用的经验来看上下文比例超过60%后就会出现降智现象。

尽可能地使用Revert和新开对话

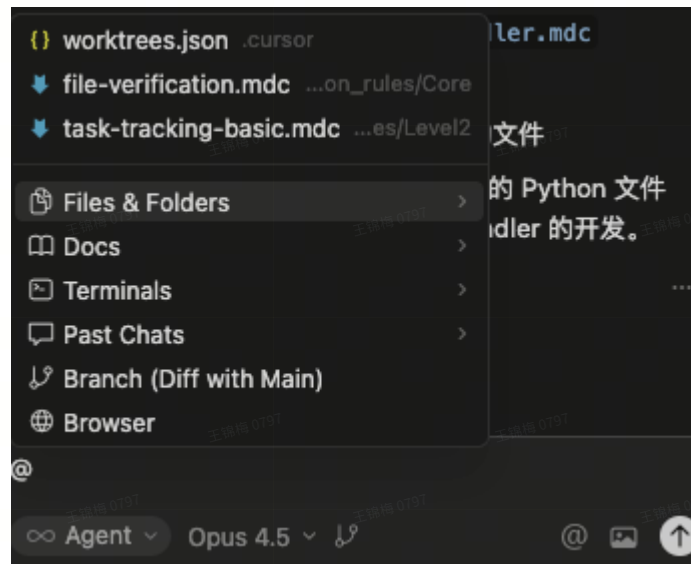
省上下文是一方面，维持上下文的简洁对模型回答质量提升也是有帮助的。因此，如果你的新问题跟历史对话关系不大，就最好新开一个对话。

在多轮对话中，如果有一个步骤出错，最好的方式也是会退到之前出错的版本，基于现状重新调整prompt和更新上下文；而不是通过对话继续修改。否则可能导致上下文中存在过多无效内容。



给出多元化的信息

我们不只可以粘代码、图片进去，还可以让模型参考网页、Git历史、当前打开的文件等，将所有关键信息给到模型会得到更加准确的输出

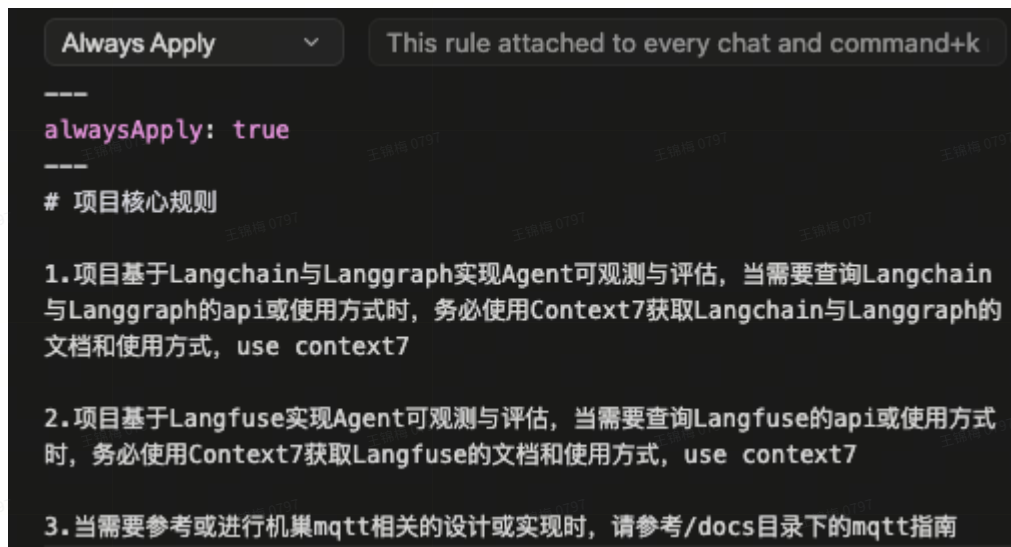
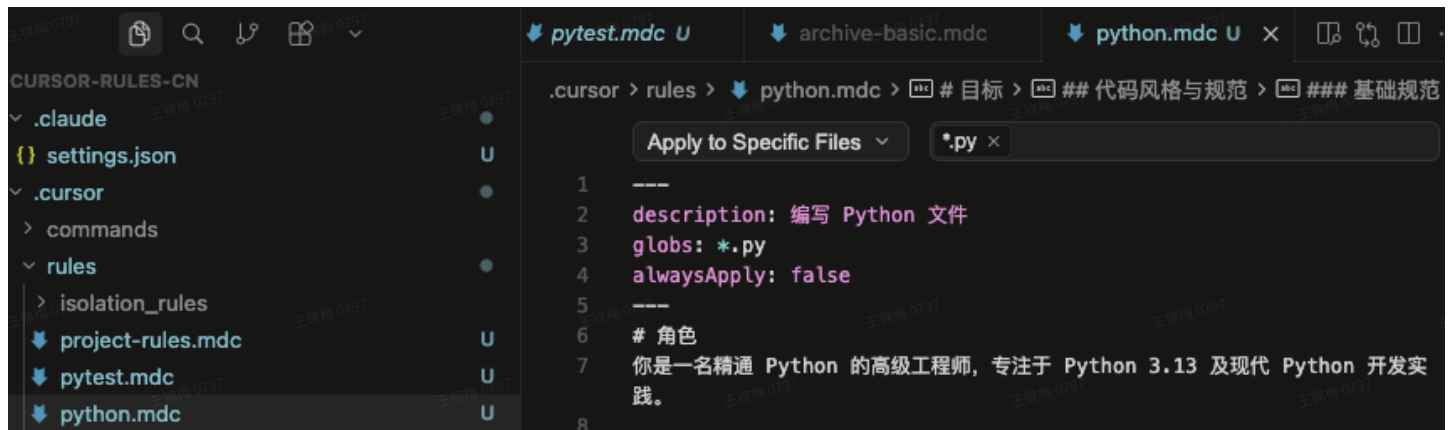


为每个项目预设Project Rule

如果一个项目没有Project Rule，可能会发现Cursor有时没有按照对应开发语言版本的规范进行编码，又或者是之前人工指出的错误又再次出现，又或者是测试用例的编写不规范。因此为整个项目预设一些开发规范或要求是有必要的，同时针对一些AI Coding出现的错误模式也需要形成知识复利添加到Rules中。大部分的编程语言或框架的通用标准可以从这个项目参考：

<https://github.com/PatrickJS/awesome-cursorrules>

- xxx语言.mdc：项目所使用编码语言的规范
- xxx语言test.mdc：项目测试用例编写规范
- project-rules.mdc：项目核心规范，用于指导什么场景下调用MCP工具或查询相关目录文档，如需求及设计文档
- project-architecture.mdc：项目核心技术架构
- xxx组件.mdc：用于总结在特定组件开发过程中的常见错误模式或约束



User Rule推荐

建议在User Rule中配置跨项目的通用规范，比如让Cursor在规划或编码过程中遇到trade off场景时主动中断让人介入决策。举一个实际遇到的例子：当Cursor在进行编码过程中明明代码已经写好了，但它突然意识到什么，又重新进行优化。

代码块

1. 凡是在方案、编码过程遇到任何争议或不确定，必须在第一时间主动告知我由我做决策。
2. 对于需要补充的信息，及时向我询问，而不是直接应用修改。
3. 每次改动基于最小范围修改原则。

使用TDD的模式开发

1. 让智能体根据预期的输入/输出对编写测试。要明确说明你在做 TDD，这样它就会避免为尚不存在的功能编写模拟实现。
2. 让智能体运行测试并确认测试确实失败。明确说明在这个阶段不要编写实现代码。
3. 在你对测试满意后提交这些测试。
4. 让智能体编写能通过测试的代码，并指示它不要修改测试。告诉它持续迭代，直到所有测试通过。
5. 在你对改动满意后提交实现代码。

采用渐进式开发

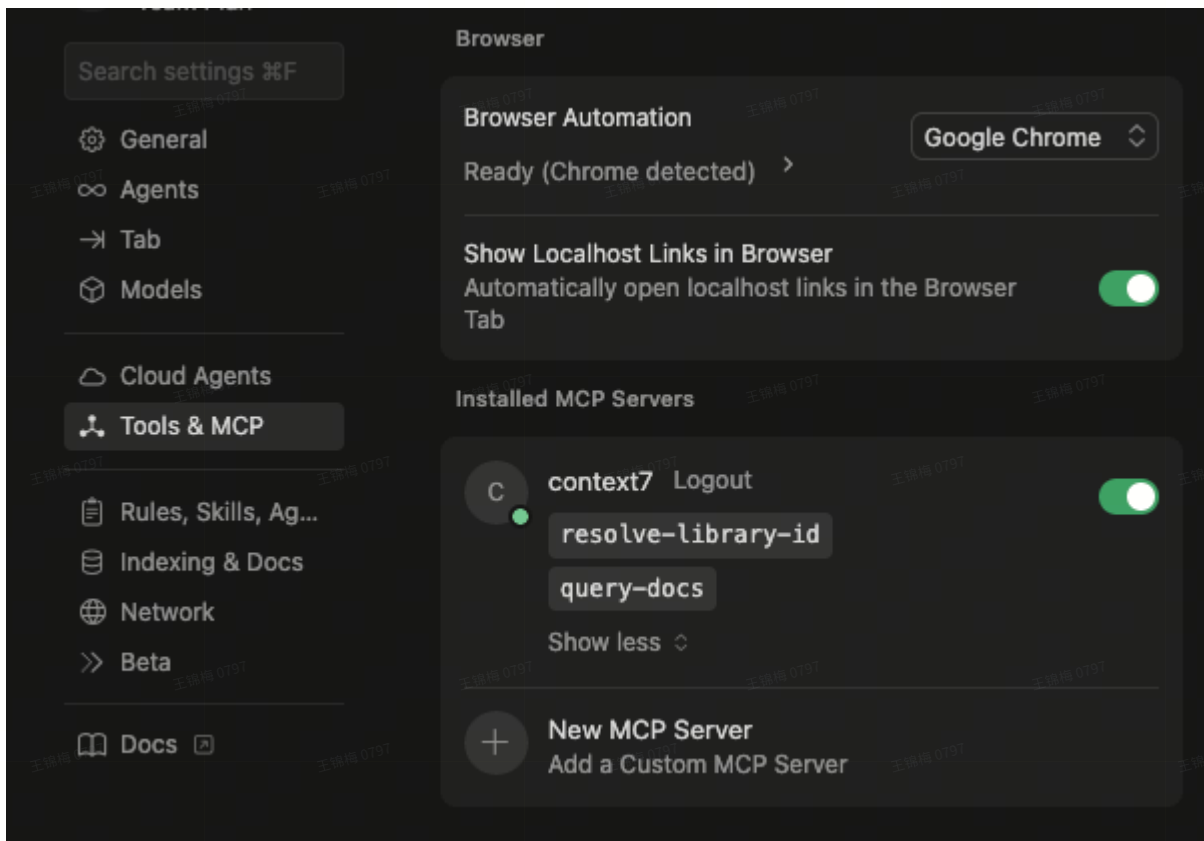
根据实践经验，不推荐输出完方案后让 AI 一口气基于方案完成需求（非常小的需求除外），需求越大代码质量越烂。我的使用方式是，跟 AI 进行**结对编程**，讨论具体的方案是什么，这个场景下的**最佳实践**是什么，拆解需求后，人工控制每一个块的代码生成。生成之后，可以咨询一下代码实现是否**优雅**，是否有**重构空间**，根据需要进行修改。

总结来说，有这些好处：

- 因为上下文窗口有限，任务粒度越小，AI 完成度越高
 - 分步骤代码量便于做 Code Review

使用合适的MCP工具

通过配置合适的MCP并将其调用时机写在Rules里，可以简化一些开发流程，比如在AI Coding过程中可能需要去查阅一些组件库或开源框架的最新使用方式，可以通过在Cursor中配置Context 7 MCP工具，并在Rules中使用use context7描述，即可自动调用MCP获取最新文档



Memory Bank

为什么需要Memory Bank

在进行一些大型项目的开发过程中，每一次开启新会话AI的记忆便都会重置，在没有记忆库的情况下，需要人工把相关的项目背景、技术架构、项目进展等喂给模型，且需要人工持续维护，那有没有一种方法，能够让AI每次执行任务时都自动加载项目相关的上下文记忆，并随任务的迭代持续更新，

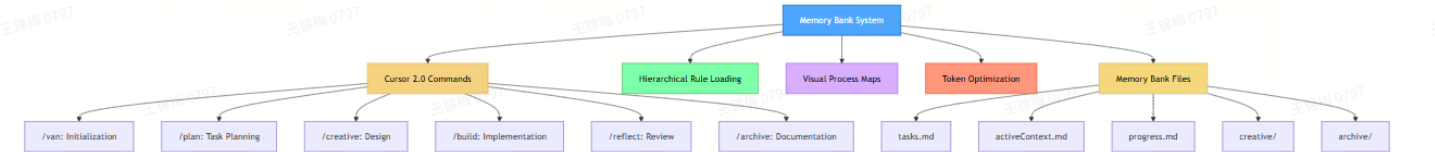
这就是2024年底Cline推出的Memory Bank功能，后续有开发者同步推出了Cursor版本，让AI能够自主沉淀知识，形成知识复利

项目地址：<https://github.com/vanzan01/cursor-memory-bank>

```
## Memory Bank 机制

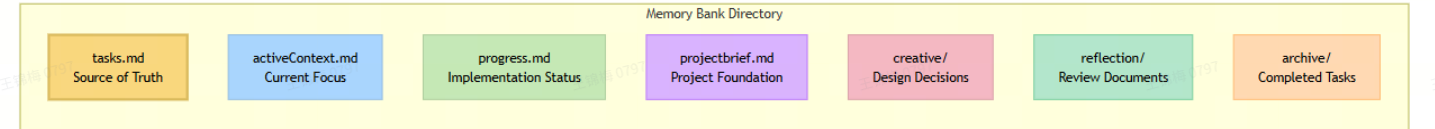
**我的记忆在每次会话后完全重置。Memory Bank 是我理解项目的唯一途径。**

🔴 关键规则：memory bank创建是强制性的 🔴
在任何模式下，memory bank必须在任何其他操作之前创建
没有验证memory bank存在，任何流程都不能继续
```



Memory Bank实施步骤

通过Cursor Command方式按照初始化、规划、创意、实施、反思、归档的方式进行迭代，人在其中的每个节点进行决策是否继续执行或补充。在进行需求的分析时会按照复杂程度分为四个等级，每个等级都有相应的决策树模型来强约束



Core Files

- **tasks.md** : Central source of truth for task tracking, checklists, and component lists
- **activeContext.md** : Maintains focus of current development phase
- **progress.md** : Tracks implementation status and observations
- **projectbrief.md** : Project foundation and context
- **productContext.md** : Product-specific context and requirements
- **systemPatterns.md** : System patterns and architectural decisions
- **techContext.md** : Technical context and technology stack

Generated Files

- **creative/creative-[feature_name].md** : Design decision documents (Level 3-4)
- **reflection/reflection-[task_id].md** : Reflection documents
- **archive/archive-[task_id].md** : Archive documents for completed tasks

Level 1: Quick Bug Fix

- **Workflow:** /van → /build → /reflect → /archive
- **Characteristics:** Single file changes, targeted fixes
- **Documentation:** Minimal, focused on the fix

Level 2: Simple Enhancement

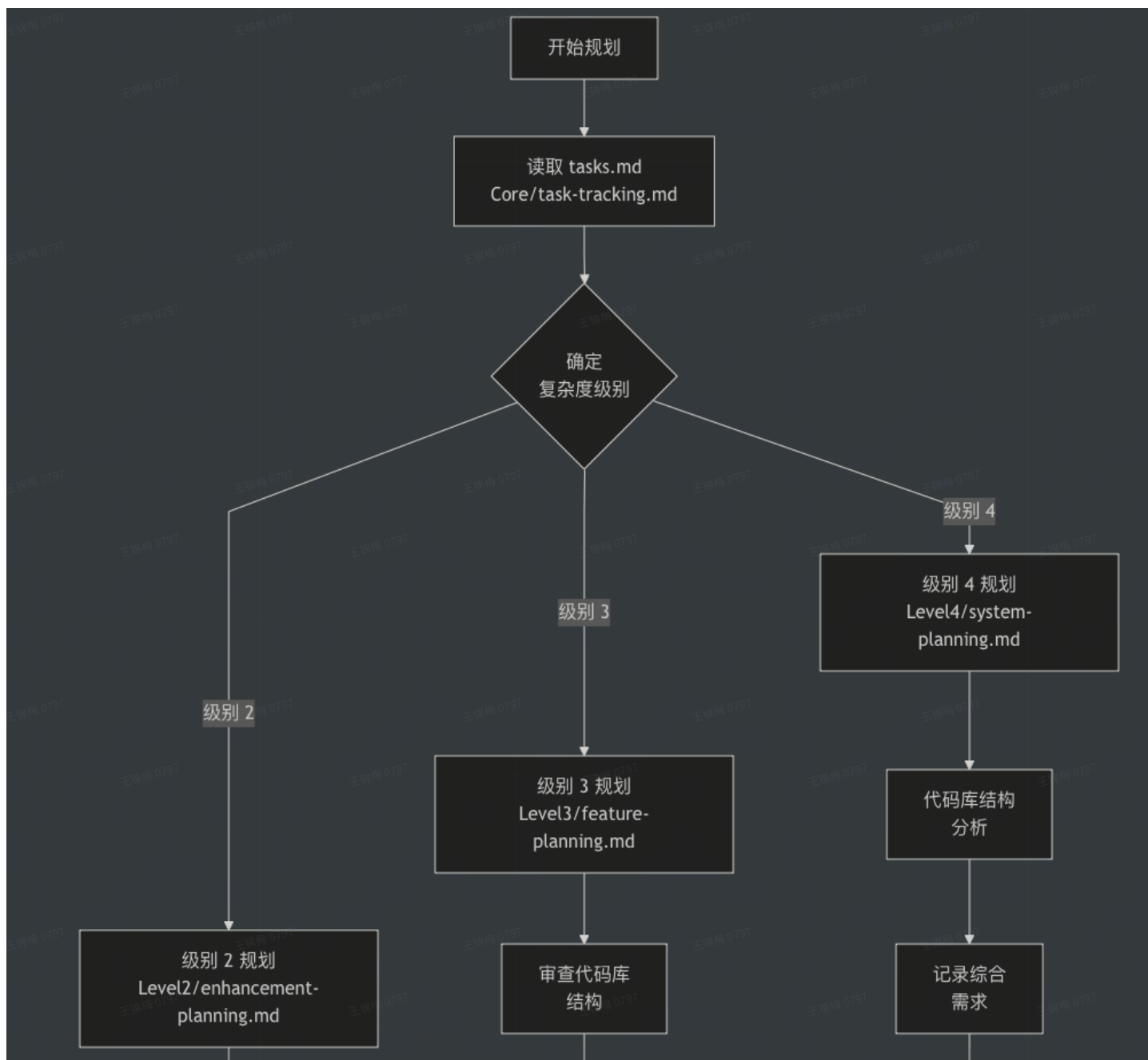
- **Workflow:** /van → /plan → /build → /reflect → /archive
- **Characteristics:** Multiple files, clear requirements
- **Documentation:** Basic plan, implementation steps

Level 3: Intermediate Feature

- **Workflow:** /van → /plan → /creative → /build → /reflect → /archive
- **Characteristics:** New components, design decisions needed
- **Documentation:** Comprehensive plan, creative phases, detailed reflection

Level 4: Complex System

- **Workflow:** /van → /plan → /creative → /build → /reflect → /archive
- **Characteristics:** Multiple subsystems, architectural decisions
- **Documentation:** Phased implementation, architectural diagrams, comprehensive archive



Memory Bank和Rules的区别

- Rules 和 Memory Bank 本质都是是上下文，用来存储记忆。
- Memory Bank 是一个逻辑功能，可以用 Rules 来实现它，比如 Memory Bank Custom Instructions 就是存储在 Rules 中。
- Memory Bank 存储的更多是一些项目的概要信息，而 Rules 存储的内容粒度会更细，比如上文中提到的一些设计要求、开发规范与编码习惯。

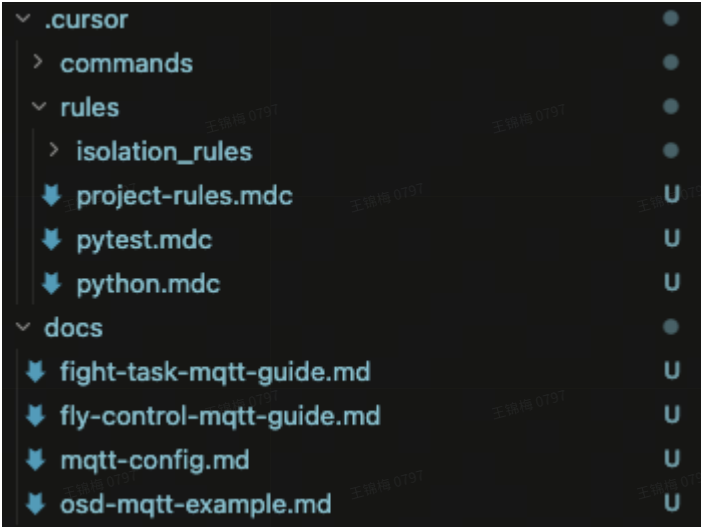
使用Memory Bank是否会导致上下文过长

此项目Rules+Memory Bank的设计是基于渐进式披露的思想，在特定的节点只会加载相关的文件，如果发现会话上下文比例占用过大，则及时开启新会话

新项目构建实战

使用Memory Bank构建机巢Mqtt Mock项目

新建Project Rules及相关文档



使用/van描述需求初始化项目

无人机机巢MQTT模拟项目 ×

视频流模块 RTMP 上传

Reflect command

+ ⌚ ...

/van 我需要开发一个无人机机巢mqtt mock的项目，模拟所有指令的接受与回复，以及osd消息的定时上报，请进行初始化

∞ Agent Opus 4.5 @ 📎 ⬆

Thought for 3s

我将按照 VAN 命令流程初始化项目。首先加载核心规则并检查项目状态。

Explored 3 directories 5 files

我已经分析了 docs 目录下的机巢 MQTT 指南文档。现在让我进行复杂度评估并初始化 Memory Bank。

复杂度分析

根据文档分析，这个 无人机机巢 MQTT Mock 项目 涉及：

维度	评估
功能模块	航线管理、指令飞行、负载控制、DRC控制、AI识别、直播、OSD上报
MQTT Topics	services/services_reply、events、drc/down/up
消息类型	20+ 种不同的 method 需要模拟
定时任务	OSD 消息定时上报（机巢+无人机状态）
状态管理	需要模拟任务执行状态变化、进度上报

复杂度判定：Level 3（中等功能）

- 涉及多个组件（MQTT 客户端、消息处理器、OSD 上报器、状态模拟器）
- 需要设计架构（消息路由、状态机、定时任务）
- 预计工期：数天到 1-2 周

现在初始化 Memory Bank 文件：

使用/plan进行方案设计

/plan

PLAN 模式：Level 3 功能规划

1. 需求分析

1.1 功能性需求

编号	需求	优先级	说明
FR-01	MQTT 连接管理	P0	连接/断开/重连 MQTT Broker
FR-02	指令接收处理	P0	订阅 services topic，解析并处理指令
FR-03	指令响应发送	P0	构建并发送 services_reply 消息
FR-04	事件上报	P0	主动上报航线进度、飞行状态等事件
FR-05	OSD 定时上报	P0	定时上报机巢和无人机 OSD
FR-06	DRC 通道支持	P1	支持 drc/down 和 drc/up 实时通道
FR-07	状态模拟	P1	模拟设备状态变化（任务执行、飞行等）
FR-08	配置化	P1	支持配置 SN、Broker、上报频率等

1.2 非功能性需求

编号	需求	指标
NFR-01	响应延迟	< 100ms
NFR-02	OSD 上报频率	可配置，默认 1Hz
NFR-03	消息格式	严格遵循 Cloud API

使用/creative进行创意阶段

/creative

开始 CREATIVE 模式，加载创意阶段规则。

Explored 1 directory 2 files

已加载创意阶段规则。现在开始对识别的两个设计问题执行创意阶段。

🎨🎨🎨 进入创意阶段：消息处理架构设计

1 问题定义

描述: 设计一个可扩展的 Handler 注册和消息路由机制，支持 39 种消息类型的处理

需求:

支持动态注册 Handler

根据 method 字段路由消息

统一的请求/响应处理流程

易于扩展新的消息类型

支持异步处理

约束:

响应延迟 < 100ms

遵循 Cloud API 消息格式

支持 services、events、drc 三种通道

2 选项探索

选项	名称	描述
A	装饰器注册模式	使用 @Handler(method=xxx) 装饰器自动注册 Handler

使用/build开始代码实施

开始 BUILD 模式。首先验证前置条件并确认实现计划。

Explored 3 files

BUILD 模式启动

前置条件验证

✓ BUILD 前置条件检查

- tasks.md 中规划完成? [✓ 是]

- 创意阶段文档存在? [✓ 是]

- creative-message-architecture.md [✓]

- creative-state-machine.md [✓]

- 实现计划明确? [✓ 是, 6个阶段]

→ 前置条件满足, 可以开始实现

阶段1: 基础框架 - 实现计划

根据规则, 我将严格按步骤执行, 先确认阶段1的实现计划:

阶段1 待创建文件

文件	职责	说明
mqtt_mock/__init__.py	包初始化	版本信息
mqtt_mock/config.py	配置管理	dataclass 定义配置
mqtt_mock/mqtt/__init__.py	mqtt 子包	-

使用/reflect进行反思

/reflect

Ran command: mkdir

\$ mkdir -p /Users/r25477/Project/cursor-rules-cn/memory-bank/reflection

Run in Sandbox

Success

reflection-MQTT-MOCK-001.md +292 -1

反思文档: MQTT-MOCK-001

任务概要

Thought for 1s

更新 Memory Bank 标记反思阶段完成。

tasks.md +1 -1

- [x] 阶段6: 集成测试

- [] REFLECT 回顾

- [x] REFLECT 回顾

- [] ARCHIVE 归档

progress.md +2 -2

CREATIVE 设计	完成	2026-01-15
BUILD 实现	待开始	-
REFLECT 回顾	待开始	-
BUILD 实现	完成	2026-01-15
REFLECT 回顾	完成	2026-01-15

使用/archive进行开发过程归档

```
/archive

Thought for 2s

我将执行归档命令。首先让我读取相关的 memory bank 文件来了解当前任务状态。

Explored 1 directory 6 files 1 search

archive-MQTT-MOCK-001.md +246 -1

# 任务归档: MQTT-MOCK-001

## 元数据

tasks.md +11 -1

- [x] REFLECT 回顾 ✓
- [ ] ARCHIVE 归档
- [x] ARCHIVE 归档 ✓

progress.md +1 -1

| REFLECT 回顾 | ✓ 完成 | 2026-01-15 |
| ARCHIVE 归档 | ⌚ 待开始 | - |
| ARCHIVE 归档 | ✓ 完成 | 2026-01-15 |

activeContext.md +18 -80
```

现有项目持续迭代

与新项目构建类似，但需要在开始功能规划前先进行Memory Bank的初始化，使用 `/van` 初始化项目Memory Bank，生成的Memory Bank需要进行人工审核，与AI多轮交互后确认项目介绍、技术架构等信息完备后再开始新功能规划

```
memory-bank > projectbrief.md > # 项目简介 - 无人机智能任务管理系统 > ## 核心组件
1 # 项目简介 - 无人机智能任务管理系统
2
3 ## 项目概述
4
5 无人机智能任务管理系统 - 基于 LangGraph 的多 Agent 协作平台。通过自然语言理解用户意图，动态规划任务流程。
6
7 ## 核心技术栈
8
9 | 技术 | 用途 |
10 |-----|-----|
11 | **FastAPI** | Web 框架，提供 REST API |
12 | **LangGraph** | Agent  workflow编排 |
13 | **Langfuse** | Agent 可观测性与评估 |
14 | **MCP** | Model Context Protocol, 外部服务交互 |
15 | **PostgreSQL** | 数据持久化 |
16
17 ## 核心架构
18
19 ...
20 用户请求 → intent_parser API → AgenticAgent → IntentRouter →
21 | SCENE → SceneAgent → SOP Workflow
22 | DYNAMIC_PLANNING → DynamicPlanner → Slot Filling → Execution Workflow
23 | CHAT → LLM 直接回复
24
25 ...
```

```
/van 参考 CLAUDE.md 初始化项目

Thought for 3s

我将执行 VAN 命令来初始化项目。首先加载核心规则并检查 memory bank 状态。

Explored 1 directory 3 files

VAN 初始化报告

1. 平台检测 ✓

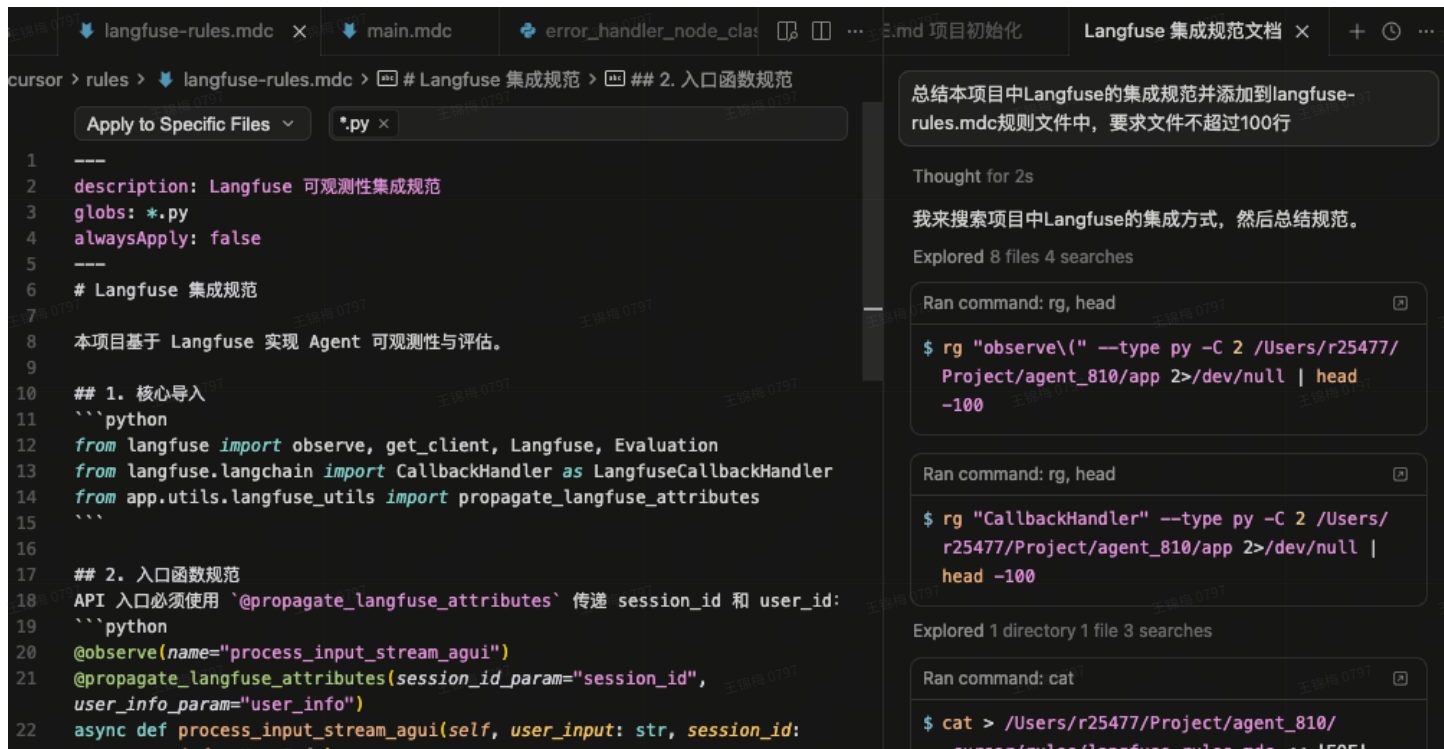
• 操作系统: macOS (darwin 24.6.0)
• Shell: zsh
• 工作区路径: /Users/r25477/Project/agent_810
• Git 仓库: 是

2. Memory Bank 验证 ⚠️

• 目录状态: memory-bank/ 存在但为空
• 操作: 需要创建核心文件

3. 项目分析 (基于 CLAUDE.md)
```

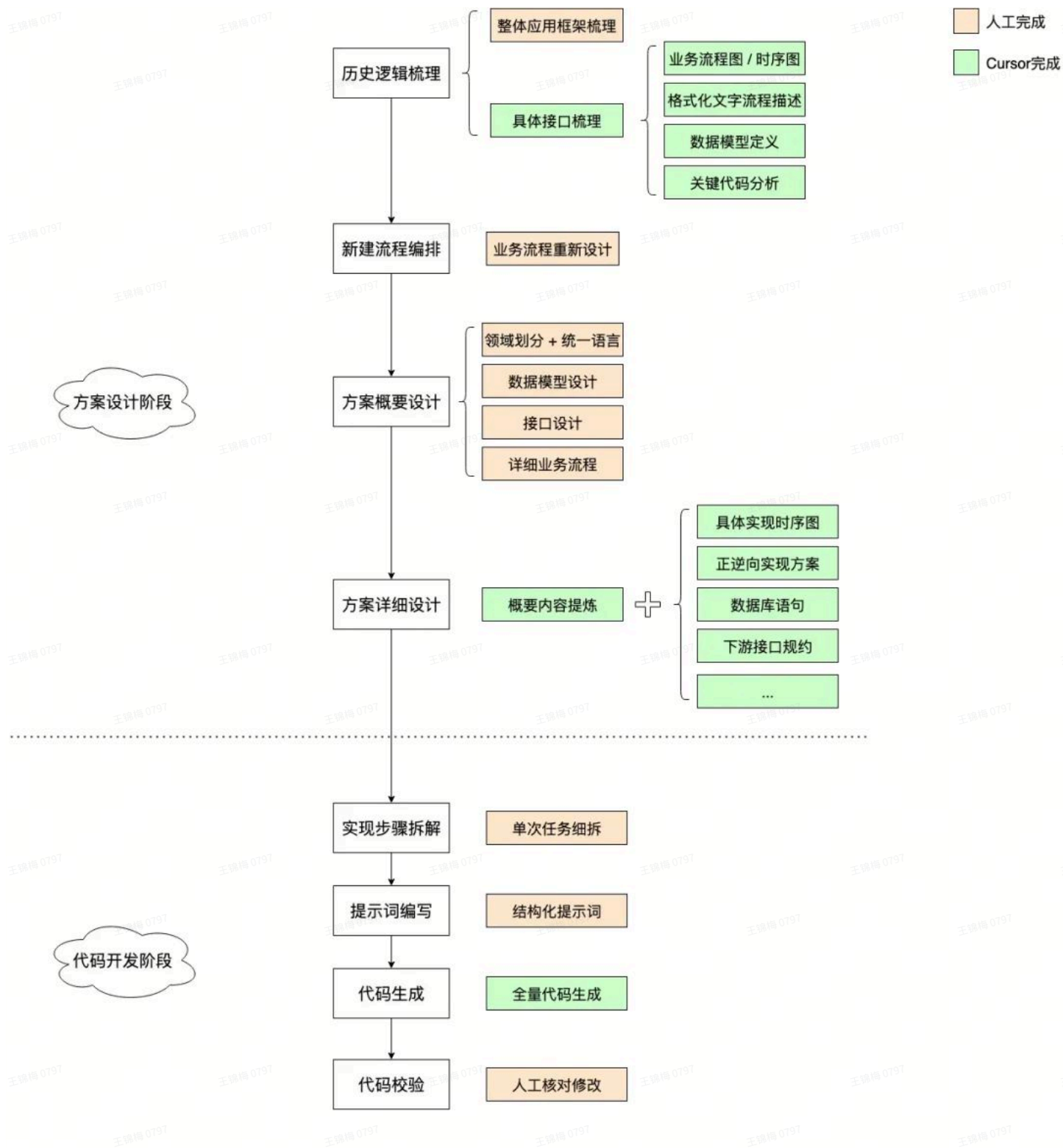
另外也可以让Cursor总结一些项目中框架或组件的开发规范，作为Project Rules



老项目重构

针对于大型项目重构的场景，当需求文档完备的前提下，可以采用上述类似的方式，先与AI进行多轮交互，生成完备的Memory Bank以及Project Rules，再根据需求进行小步迭代

如果整个项目都缺乏详细文档，那么就先需要通过AI进行所有接口的梳理，通过接口及其内部逻辑逆向出需求设计，或使用类似DeepWiki的工具构建项目Wiki，然后再进行架构的设计及分步实现



总结与展望

任何AI Coding工具都不是银弹，Scaling Laws意味着模型的推理能力和上下文长度在未来会不断增强，但如果无法给大模型提供准确完备的上下文知识，再简单的场景也可能实现上达不到预期，因此知识的沉淀、知识的复利也是非常重要的，这需要人与AI来共同构建